

pickle — Python object serialization

Source code: [Lib/pickle.py](#)

The `pickle` module implements binary protocols for serializing and de-serializing a Python object structure. “Pickling” is the process whereby a Python object hierarchy is converted into a byte stream, and “unpickling” is the inverse operation, whereby a byte stream (from a [binary file](#) or [bytes-like object](#)) is converted back into an object hierarchy. Pickling (and unpickling) is alternatively known as “serialization”, “marshalling”, [\[1\]](#) or “flattening”; however, to avoid confusion, the terms used here are “pickling” and “unpickling”.

Warning: The `pickle` module **is not secure**. Only unpickle data you trust.

It is possible to construct malicious pickle data which will **execute arbitrary code during unpickling**. Never unpickle data that could have come from an untrusted source, or that could have been tampered with.

Consider signing data with [hmac](#) if you need to ensure that it has not been tampered with.

Safer serialization formats such as [json](#) may be more appropriate if you are processing untrusted data. See [Comparison with json](#).

Relationship to other Python modules

Comparison with `marshal`

Python has a more primitive serialization module called [marshal](#), but in general `pickle` should always be the preferred way to serialize Python objects. `marshal` exists primarily to support Python’s `.pyc` files.

The `pickle` module differs from [marshal](#) in several significant ways:

- [marshal](#) cannot be used to serialize user-defined classes and their instances. `pickle` can save and restore class instances transparently, however the class definition must be importable and live in the same module as when the object was pickled.
- The [marshal](#) serialization format is not guaranteed to be portable across Python versions. Because its primary job in life is to support `.pyc` files, the Python implementers reserve the right to change the serialization format in non-backwards compatible ways should the need arise. The `pickle` serialization format is guaranteed to be backwards compatible across Python releases provided a compatible pickle protocol is chosen and pickling and unpickling code deals with Python 2 to Python 3 type differences if your data is crossing that unique breaking change language boundary.

Comparison with `json`

There are fundamental differences between the pickle protocols and [JSON \(JavaScript Object Notation\)](#):

- JSON is a text serialization format (it outputs unicode text, although most of the time it is then encoded to utf-8), while pickle is a binary serialization format;
- JSON is human-readable, while pickle is not;

- JSON is interoperable and widely used outside of the Python ecosystem, while pickle is Python-specific;
- JSON, by default, can only represent a subset of the Python built-in types, and no custom classes; pickle can represent an extremely large number of Python types (many of them automatically, by clever usage of Python's introspection facilities; complex cases can be tackled by implementing [specific object APIs](#));
- Unlike pickle, deserializing untrusted JSON does not in itself create an arbitrary code execution vulnerability.

See also: The [json](#) module: a standard library module allowing JSON serialization and deserialization.

Data stream format

The data format used by `pickle` is Python-specific. This has the advantage that there are no restrictions imposed by external standards such as JSON (which can't represent pointer sharing); however it means that non-Python programs may not be able to reconstruct pickled Python objects.

By default, the `pickle` data format uses a relatively compact binary representation. If you need optimal size characteristics, you can efficiently [compress](#) pickled data.

The module [pickletools](#) contains tools for analyzing data streams generated by `pickle`. `pickletools` source code has extensive comments about opcodes used by pickle protocols.

There are currently 6 different protocols which can be used for pickling. The higher the protocol used, the more recent the version of Python needed to read the pickle produced.

- Protocol version 0 is the original "human-readable" protocol and is backwards compatible with earlier versions of Python.
- Protocol version 1 is an old binary format which is also compatible with earlier versions of Python.
- Protocol version 2 was introduced in Python 2.3. It provides much more efficient pickling of [new-style classes](#). Refer to [PEP 307](#) for information about improvements brought by protocol 2.
- Protocol version 3 was added in Python 3.0. It has explicit support for [bytes](#) objects and cannot be unpickled by Python 2.x. This was the default protocol in Python 3.0–3.7.
- Protocol version 4 was added in Python 3.4. It adds support for very large objects, pickling more kinds of objects, and some data format optimizations. This was the default protocol in Python 3.8–3.13. Refer to [PEP 3154](#) for information about improvements brought by protocol 4.
- Protocol version 5 was added in Python 3.8. It adds support for out-of-band data and speedup for in-band data. It is the default protocol starting with Python 3.14. Refer to [PEP 574](#) for information about improvements brought by protocol 5.

Note: Serialization is a more primitive notion than persistence; although `pickle` reads and writes file objects, it does not handle the issue of naming persistent objects, nor the (even more complicated) issue of concurrent access to persistent objects. The `pickle` module can transform a complex object into a byte stream and it can transform the byte stream into an object with the same internal structure. Perhaps the most obvious thing to do with these byte streams is to write them onto a file, but it is also conceivable to send them across a network or store them in a database. The [shelve](#) module provides a simple interface to pickle and unpickle objects on DBM-style database files.

Module Interface

To serialize an object hierarchy, you simply call the [dumps\(\)](#) function. Similarly, to de-serialize a data stream, you call the [loads\(\)](#) function. However, if you want more control over serialization and de-serialization, you can create a [Pickler](#) or an [Unpickler](#) object, respectively.

The pickle module provides the following constants:

`pickle.HIGHEST_PROTOCOL`

An integer, the highest [protocol version](#) available. This value can be passed as a *protocol* value to functions [dump\(\)](#) and [dumps\(\)](#) as well as the [Pickler](#) constructor.

`pickle.DEFAULT_PROTOCOL`

An integer, the default [protocol version](#) used for pickling. May be less than [HIGHEST_PROTOCOL](#). Currently the default protocol is 5, introduced in Python 3.8 and incompatible with previous versions. This version introduces support for out-of-band buffers, where [PEP 3118](#)-compatible data can be transmitted separately from the main pickle stream.

Changed in version 3.0: The default protocol is 3.

Changed in version 3.8: The default protocol is 4.

Changed in version 3.14: The default protocol is 5.

The pickle module provides the following functions to make the pickling process more convenient:

`pickle.dump(obj, file, protocol=None, *, fix_imports=True, buffer_callback=None)`

Write the pickled representation of the object *obj* to the open [file object](#) *file*. This is equivalent to `Pickler(file, protocol).dump(obj)`.

Arguments *file*, *protocol*, *fix_imports* and *buffer_callback* have the same meaning as in the [Pickler](#) constructor.

Changed in version 3.8: The *buffer_callback* argument was added.

`pickle.dumps(obj, protocol=None, *, fix_imports=True, buffer_callback=None)`

Return the pickled representation of the object *obj* as a [bytes](#) object, instead of writing it to a file.

Arguments *protocol*, *fix_imports* and *buffer_callback* have the same meaning as in the [Pickler](#) constructor.

Changed in version 3.8: The *buffer_callback* argument was added.

`pickle.load(file, *, fix_imports=True, encoding='ASCII', errors='strict', buffers=None)`

Read the pickled representation of an object from the open [file object](#) *file* and return the reconstituted object hierarchy specified therein. This is equivalent to `Unpickler(file).load()`.

The protocol version of the pickle is detected automatically, so no protocol argument is needed. Bytes past the pickled representation of the object are ignored.

Arguments *file*, *fix_imports*, *encoding*, *errors*, *strict* and *buffers* have the same meaning as in the [Unpickler](#) constructor.

Changed in version 3.8: The *buffers* argument was added.

```
pickle.loads(data, /, *, fix_imports=True, encoding='ASCII', errors='strict', buffers=None)
```

Return the reconstituted object hierarchy of the pickled representation *data* of an object. *data* must be a [bytes-like object](#).

The protocol version of the pickle is detected automatically, so no protocol argument is needed. Bytes past the pickled representation of the object are ignored.

Arguments *fix_imports*, *encoding*, *errors*, *strict* and *buffers* have the same meaning as in the [Unpickler](#) constructor.

Changed in version 3.8: The *buffers* argument was added.

The pickle module defines three exceptions:

exception pickle.PickleError

Common base class for the other pickling exceptions. It inherits from [Exception](#).

exception pickle.PicklingError

Error raised when an unpicklable object is encountered by [Pickler](#). It inherits from [PickleError](#).

Refer to [What can be pickled and unpickled?](#) to learn what kinds of objects can be pickled.

exception pickle.UnpicklingError

Error raised when there is a problem unpickling an object, such as a data corruption or a security violation. It inherits from [PickleError](#).

Note that other exceptions may also be raised during unpickling, including (but not necessarily limited to) `AttributeError`, `EOFError`, `ImportError`, and `IndexError`.

The pickle module exports three classes, [Pickler](#), [Unpickler](#) and [PickleBuffer](#):

```
class pickle.Pickler(file, protocol=None, *, fix_imports=True, buffer_callback=None)
```

This takes a binary file for writing a pickle data stream.

The optional *protocol* argument, an integer, tells the pickler to use the given protocol; supported protocols are 0 to [HIGHEST_PROTOCOL](#). If not specified, the default is [DEFAULT_PROTOCOL](#). If a negative number is specified, `HIGHEST_PROTOCOL` is selected.

The *file* argument must have a `write()` method that accepts a single bytes argument. It can thus be an on-disk file opened for binary writing, an [io.BytesIO](#) instance, or any other custom object that meets this interface.

If *fix_imports* is true and *protocol* is less than 3, pickle will try to map the new Python 3 names to the old module names used in Python 2, so that the pickle data stream is readable with Python 2.

If *buffer_callback* is None (the default), buffer views are serialized into *file* as part of the pickle stream.

If *buffer_callback* is not None, then it can be called any number of times with a buffer view. If the callback returns a false value (such as None), the given buffer is [out-of-band](#); otherwise the buffer is serialized in-band, i.e. inside the pickle stream.

It is an error if *buffer_callback* is not None and *protocol* is None or smaller than 5.

Changed in version 3.8: The `buffer_callback` argument was added.

dump(obj)

Write the pickled representation of `obj` to the open file object given in the constructor.

persistent_id(obj)

Do nothing by default. This exists so a subclass can override it.

If `persistent_id()` returns `None`, `obj` is pickled as usual. Any other value causes `Pickler` to emit the returned value as a persistent ID for `obj`. The meaning of this persistent ID should be defined by [Unpickler.persistent_load\(\)](#). Note that the value returned by `persistent_id()` cannot itself have a persistent ID.

See [Persistence of External Objects](#) for details and examples of uses.

Changed in version 3.13: Add the default implementation of this method in the C implementation of `Pickler`.

dispatch_table

A pickler object's dispatch table is a registry of *reduction functions* of the kind which can be declared using [copyreg.pickle\(\)](#). It is a mapping whose keys are classes and whose values are reduction functions. A reduction function takes a single argument of the associated class and should conform to the same interface as a [__reduce__\(\)](#) method.

By default, a pickler object will not have a [dispatch_table](#) attribute, and it will instead use the global dispatch table managed by the [copyreg](#) module. However, to customize the pickling for a specific pickler object one can set the `dispatch_table` attribute to a dict-like object. Alternatively, if a subclass of `Pickler` has a `dispatch_table` attribute then this will be used as the default dispatch table for instances of that class.

See [Dispatch Tables](#) for usage examples.

Added in version 3.3.

reducer_override(obj)

Special reducer that can be defined in `Pickler` subclasses. This method has priority over any reducer in the [dispatch_table](#). It should conform to the same interface as a [__reduce__\(\)](#) method, and can optionally return [NotImplemented](#) to fallback on `dispatch_table`-registered reducers to pickle `obj`.

For a detailed example, see [Custom Reduction for Types, Functions, and Other Objects](#).

Added in version 3.8.

fast

Deprecated. Enable fast mode if set to a true value. The fast mode disables the usage of memo, therefore speeding the pickling process by not generating superfluous PUT opcodes. It should not be used with self-referential objects, doing otherwise will cause `Pickler` to recurse infinitely.

Use [pickletools.optimize\(\)](#) if you need more compact pickles.

`clear_memo()`

Clears the pickler's "memo".

The memo is the data structure that remembers which objects the pickler has already seen, so that shared or recursive objects are pickled by reference and not by value. This method is useful when re-using picklers.

```
class pickle.Unpickler(file, *, fix_imports=True, encoding='ASCII', errors='strict', buffers=None)
```

This takes a binary file for reading a pickle data stream.

The protocol version of the pickle is detected automatically, so no protocol argument is needed.

The argument *file* must have three methods, a `read()` method that takes an integer argument, a `readinto()` method that takes a buffer argument and a `readline()` method that requires no arguments, as in the [io.BufferedIOBase](#) interface. Thus *file* can be an on-disk file opened for binary reading, an [io.BytesIO](#) object, or any other custom object that meets this interface.

The optional arguments *fix_imports*, *encoding* and *errors* are used to control compatibility support for pickle stream generated by Python 2. If *fix_imports* is true, pickle will try to map the old Python 2 names to the new names used in Python 3. The *encoding* and *errors* tell pickle how to decode 8-bit string instances pickled by Python 2; these default to 'ASCII' and 'strict', respectively. The *encoding* can be 'bytes' to read these 8-bit string instances as bytes objects. Using *encoding*='latin1' is required for unpickling NumPy arrays and instances of [datetime](#), [date](#) and [time](#) pickled by Python 2.

If *buffers* is None (the default), then all data necessary for deserialization must be contained in the pickle stream. This means that the *buffer_callback* argument was None when a [Pickler](#) was instantiated (or when [dump\(\)](#) or [dumps\(\)](#) was called).

If *buffers* is not None, it should be an iterable of buffer-enabled objects that is consumed each time the pickle stream references an [out-of-band](#) buffer view. Such buffers have been given in order to the *buffer_callback* of a Pickler object.

Changed in version 3.8: The *buffers* argument was added.

`load()`

Read the pickled representation of an object from the open file object given in the constructor, and return the reconstituted object hierarchy specified therein. Bytes past the pickled representation of the object are ignored.

`persistent_load(pid)`

Raise an [UnpicklingError](#) by default.

If defined, `persistent_load()` should return the object specified by the persistent ID *pid*. If an invalid persistent ID is encountered, an [UnpicklingError](#) should be raised.

See [Persistence of External Objects](#) for details and examples of uses.

Changed in version 3.13: Add the default implementation of this method in the C implementation of Unpickler.

find_class(*module*, *name*)

Import *module* if necessary and return the object called *name* from it, where the *module* and *name* arguments are [str](#) objects. Note, unlike its name suggests, `find_class()` is also used for finding functions.

Subclasses may override this to gain control over what type of objects and how they can be loaded, potentially reducing security risks. Refer to [Restricting Globals](#) for details.

Raises an [auditing event](#) `pickle.find_class` with arguments `module`, `name`.

class pickle.PickleBuffer(*buffer*)

A wrapper for a buffer representing picklable data. *buffer* must be a [buffer-providing](#) object, such as a [bytes-like object](#) or a N-dimensional array.

PickleBuffer is itself a buffer provider, therefore it is possible to pass it to other APIs expecting a buffer-providing object, such as [memoryview](#).

PickleBuffer objects can only be serialized using pickle protocol 5 or higher. They are eligible for [out-of-band serialization](#).

Added in version 3.8.

raw()

Return a [memoryview](#) of the memory area underlying this buffer. The returned object is a one-dimensional, C-contiguous memoryview with format B (unsigned bytes). [BufferError](#) is raised if the buffer is neither C- nor Fortran-contiguous.

release()

Release the underlying buffer exposed by the PickleBuffer object.

What can be pickled and unpickled?

The following types can be pickled:

- built-in constants (None, True, False, Ellipsis, and [NotImplemented](#));
- integers, floating-point numbers, complex numbers;
- strings, bytes, bytearray;
- tuples, lists, sets, and dictionaries containing only picklable objects;
- functions (built-in and user-defined) accessible from the top level of a module (using [def](#), not [lambda](#));
- classes accessible from the top level of a module;
- instances of such classes for which the result of calling [__getstate__\(\)](#) is picklable (see section [Pickling, Class Instances](#) for details).

Attempts to pickle unpicklable objects will raise the [PicklingError](#) exception; when this happens, an unspecified number of bytes may have already been written to the underlying file. Trying to pickle a highly recursive data structure may exceed the maximum recursion depth, a [RecursionError](#) will be raised in this case. You can carefully raise this limit with [sys.setrecursionlimit\(\)](#).

Note that functions (built-in and user-defined) are pickled by fully [qualified name](#), not by value. [\[2\]](#) This means that only the function name is pickled, along with the name of the containing module and classes. Neither the function's code, nor any of its function attributes are pickled. Thus the defining module must be importable in the unpickling environment, and the module must contain the named object, otherwise an exception will be raised. [\[3\]](#)

Similarly, classes are pickled by fully qualified name, so the same restrictions in the unpickling environment apply. Note that none of the class's code or data is pickled, so in the following example the class attribute `attr` is not restored in the unpickling environment:

```
class Foo:
    attr = 'A class attribute'

picklestring = pickle.dumps(Foo)
```

These restrictions are why picklable functions and classes must be defined at the top level of a module.

Similarly, when class instances are pickled, their class's code and data are not pickled along with them. Only the instance data are pickled. This is done on purpose, so you can fix bugs in a class or add methods to the class and still load objects that were created with an earlier version of the class. If you plan to have long-lived objects that will see many versions of a class, it may be worthwhile to put a version number in the objects so that suitable conversions can be made by the class's [__setstate__\(\)](#) method.

Pickling Class Instances

In this section, we describe the general mechanisms available to you to define, customize, and control how class instances are pickled and unpickled.

In most cases, no additional code is needed to make instances picklable. By default, pickle will retrieve the class and the attributes of an instance via introspection. When a class instance is unpickled, its [__init__\(\)](#) method is usually *not* invoked. The default behaviour first creates an uninitialized instance and then restores the saved attributes. The following code shows an implementation of this behaviour:

```
def save(obj):
    return (obj.__class__, obj.__dict__)

def restore(cls, attributes):
    obj = cls.__new__(cls)
    obj.__dict__.update(attributes)
    return obj
```

Classes can alter the default behaviour by providing one or several special methods:

object.[__getnewargs_ex__\(\)](#)

In protocols 2 and newer, classes that implement the [__getnewargs_ex__\(\)](#) method can dictate the values passed to the [__new__\(\)](#) method upon unpickling. The method must return a pair (`args`, `kwargs`) where `args` is a tuple of positional arguments and `kwargs` a dictionary of named arguments for constructing the object. Those will be passed to the [__new__\(\)](#) method upon unpickling.

You should implement this method if the [__new__\(\)](#) method of your class requires keyword-only arguments. Otherwise, it is recommended for compatibility to implement [__getnewargs__\(\)](#).

Changed in version 3.6: `__getnewargs_ex__()` is now used in protocols 2 and 3.

`object.__getnewargs__()`

This method serves a similar purpose as `__getnewargs_ex__()`, but supports only positional arguments. It must return a tuple of arguments `args` which will be passed to the `__new__()` method upon unpickling.

`__getnewargs__()` will not be called if `__getnewargs_ex__()` is defined.

Changed in version 3.6: Before Python 3.6, `__getnewargs__()` was called instead of `__getnewargs_ex__()` in protocols 2 and 3.

`object.__getstate__()`

Classes can further influence how their instances are pickled by overriding the method `__getstate__()`. It is called and the returned object is pickled as the contents for the instance, instead of a default state.

There are several cases:

- For a class that has no instance `__dict__` and no `__slots__`, the default state is `None`.
- For a class that has an instance `__dict__` and no `__slots__`, the default state is `self.__dict__`.
- For a class that has an instance `__dict__` and `__slots__`, the default state is a tuple consisting of two dictionaries: `self.__dict__`, and a dictionary mapping slot names to slot values. Only slots that have a value are included in the latter.
- For a class that has `__slots__` and no instance `__dict__`, the default state is a tuple whose first item is `None` and whose second item is a dictionary mapping slot names to slot values described in the previous bullet.

Changed in version 3.11: Added the default implementation of the `__getstate__()` method in the `object` class.

`object.__setstate__(state)`

Upon unpickling, if the class defines `__setstate__()`, it is called with the unpickled state. In that case, there is no requirement for the state object to be a dictionary. Otherwise, the pickled state must be a dictionary and its items are assigned to the new instance's dictionary.

Note: If `__reduce__()` returns a state with value `None` at pickling, the `__setstate__()` method will not be called upon unpickling.

Refer to the section [Handling Stateful Objects](#) for more information about how to use the methods `__getstate__()` and `__setstate__()`.

Note: At unpickling time, some methods like `__getattr__()`, `__getattribute__()`, or `__setattr__()` may be called upon the instance. In case those methods rely on some internal invariant being true, the type should implement `__new__()` to establish such an invariant, as `__init__()` is not called when unpickling an instance.

As we shall see, pickle does not use directly the methods described above. In fact, these methods are part of the copy protocol which implements the `__reduce__()` special method. The copy protocol provides a unified interface for retrieving the data necessary for pickling and copying objects. [\[4\]](#)

Although powerful, implementing `__reduce__()` directly in your classes is error prone. For this reason, class designers should use the high-level interface (i.e., `__getnewargs_ex__()`, `__getstate__()` and `__setstate__()`) whenever possible. We will show, however, cases where using `__reduce__()` is the only option or leads to more efficient pickling or both.

object.`__reduce__()`

The interface is currently defined as follows. The `__reduce__()` method takes no argument and shall return either a string or preferably a tuple (the returned object is often referred to as the “reduce value”).

If a string is returned, the string should be interpreted as the name of a global variable. It should be the object’s local name relative to its module; the pickle module searches the module namespace to determine the object’s module: for a given `obj` to be pickled, the `__module__` attribute is looked up on `obj` directly, which falls back to a lookup on the type of `obj` if no `__module__` instance attribute is set. This behaviour is typically useful for singletons.

When a tuple is returned, it must be between two and six items long. Optional items can either be omitted, or `None` can be provided as their value. The semantics of each item are in order:

- A callable object that will be called to create the initial version of the object.
- A tuple of arguments for the callable object. An empty tuple must be given if the callable does not accept any argument.
- Optionally, the object’s state, which will be passed to the object’s `__setstate__()` method as previously described. If the object has no such method then, the value must be a dictionary and it will be added to the object’s `__dict__` attribute.
- Optionally, an iterator (and not a sequence) yielding successive items. These items will be appended to the object either using `obj.append(item)` or, in batch, using `obj.extend(list_of_items)`. This is primarily used for list subclasses, but may be used by other classes as long as they have `append()` and `extend()` methods with the appropriate signature. (Whether `append()` or `extend()` is used depends on which pickle protocol version is used as well as the number of items to append, so both must be supported.)
- Optionally, an iterator (not a sequence) yielding successive key-value pairs. These items will be stored to the object using `obj[key] = value`. This is primarily used for dictionary subclasses, but may be used by other classes as long as they implement `__setitem__()`.
- Optionally, a callable with a `(obj, state)` signature. This callable allows the user to programmatically control the state-updating behavior of a specific object, instead of using `obj`’s static `__setstate__()` method. If not `None`, this callable will have priority over `obj`’s `__setstate__()`.

Added in version 3.8: The optional sixth tuple item, `(obj, state)`, was added.

object.`__reduce_ex__(protocol)`

Alternatively, a `__reduce_ex__()` method may be defined. The only difference is this method should take a single integer argument, the protocol version. When defined, pickle will prefer it over the `__reduce__()` method. In addition, `__reduce__()` automatically becomes a synonym for the extended version. The main use for this method is to provide backwards-compatible reduce values for older Python releases.

Persistence of External Objects

For the benefit of object persistence, the `pickle` module supports the notion of a reference to an object outside the pickled data stream. Such objects are referenced by a persistent ID, which should be either a string of alphanumeric characters (for protocol 0) [5] or just an arbitrary object (for any newer protocol).

The resolution of such persistent IDs is not defined by the `pickle` module; it will delegate this resolution to the user-defined methods on the pickler and unpickler, [`persistent\_id\(\)`](#) and [`persistent\_load\(\)`](#) respectively.

To pickle objects that have an external persistent ID, the pickler must have a custom [`persistent\_id\(\)`](#) method that takes an object as an argument and returns either `None` or the persistent ID for that object. When `None` is returned, the pickler simply pickles the object as normal. When a persistent ID string is returned, the pickler will pickle that object, along with a marker so that the unpickler will recognize it as a persistent ID.

To unpickle external objects, the unpickler must have a custom [`persistent\_load\(\)`](#) method that takes a persistent ID object and returns the referenced object.

Here is a comprehensive example presenting how persistent ID can be used to pickle external objects by reference.

```
# Simple example presenting how persistent ID can be used to pickle
# external objects by reference.

import pickle
import sqlite3
from collections import namedtuple

# Simple class representing a record in our database.
MemoRecord = namedtuple("MemoRecord", "key, task")

class DBPickler(pickle.Pickler):

    def persistent_id(self, obj):
        # Instead of pickling MemoRecord as a regular class instance, we emit a
        # persistent ID.
        if isinstance(obj, MemoRecord):
            # Here, our persistent ID is simply a tuple, containing a tag and a
            # key, which refers to a specific record in the database.
            return ("MemoRecord", obj.key)
        else:
            # If obj does not have a persistent ID, return None. This means obj
            # needs to be pickled as usual.
            return None

class DBUnpickler(pickle.Unpickler):

    def __init__(self, file, connection):
        super().__init__(file)
        self.connection = connection

    def persistent_load(self, pid):
        # This method is invoked whenever a persistent ID is encountered.
        # Here, pid is the tuple returned by DBPickler.
        cursor = self.connection.cursor()
        type_tag, key_id = pid
        if type_tag == "MemoRecord":
            # Fetch the referenced record from the database and return it.
            cursor.execute("SELECT * FROM memos WHERE key=?", (str(key_id),))
```

```

        key, task = cursor.fetchone()
        return MemoRecord(key, task)
    else:
        # Always raises an error if you cannot return the correct object.
        # Otherwise, the unpickler will think None is the object referenced
        # by the persistent ID.
        raise pickle.UnpicklingError("unsupported persistent object")

def main():
    import io
    import pprint

    # Initialize and populate our database.
    conn = sqlite3.connect(":memory:")
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE memos(key INTEGER PRIMARY KEY, task TEXT)")
    tasks = (
        'give food to fish',
        'prepare group meeting',
        'fight with a zebra',
    )
    for task in tasks:
        cursor.execute("INSERT INTO memos VALUES(NULL, ?)", (task,))

    # Fetch the records to be pickled.
    cursor.execute("SELECT * FROM memos")
    memos = [MemoRecord(key, task) for key, task in cursor]
    # Save the records using our custom DBPickler.
    file = io.BytesIO()
    DBPickler(file).dump(memos)

    print("Pickled records:")
    pprint.pprint(memos)

    # Update a record, just for good measure.
    cursor.execute("UPDATE memos SET task='learn italian' WHERE key=1")

    # Load the records from the pickle data stream.
    file.seek(0)
    memos = DBUnpickler(file, conn).load()

    print("Unpickled records:")
    pprint.pprint(memos)

if __name__ == '__main__':
    main()

```

Dispatch Tables

If one wants to customize pickling of some classes without disturbing any other code which depends on pickling, then one can create a pickler with a private dispatch table.

The global dispatch table managed by the [copyreg](#) module is available as `copyreg.dispatch_table`. Therefore, one may choose to use a modified copy of `copyreg.dispatch_table` as a private dispatch table.

For example

```

f = io.BytesIO()
p = pickle.Pickler(f)

```

```
p.dispatch_table = copyreg.dispatch_table.copy()
p.dispatch_table[SomeClass] = reduce_SomeClass
```

creates an instance of [pickle.Pickler](#) with a private dispatch table which handles the `SomeClass` class specially. Alternatively, the code

```
class MyPickler(pickle.Pickler):
    dispatch_table = copyreg.dispatch_table.copy()
    dispatch_table[SomeClass] = reduce_SomeClass
f = io.BytesIO()
p = MyPickler(f)
```

does the same but all instances of `MyPickler` will by default share the private dispatch table. On the other hand, the code

```
copyreg.pickle(SomeClass, reduce_SomeClass)
f = io.BytesIO()
p = pickle.Pickler(f)
```

modifies the global dispatch table shared by all users of the [copyreg](#) module.

Handling Stateful Objects

Here's an example that shows how to modify pickling behavior for a class. The `TextReader` class below opens a text file, and returns the line number and line contents each time its `readline()` method is called. If a `TextReader` instance is pickled, all attributes *except* the file object member are saved. When the instance is unpickled, the file is reopened, and reading resumes from the last location. The `__setstate__()` and `__getstate__()` methods are used to implement this behavior.

```
class TextReader:
    """Print and number lines in a text file."""

    def __init__(self, filename):
        self.filename = filename
        self.file = open(filename)
        self.lineno = 0

    def readline(self):
        self.lineno += 1
        line = self.file.readline()
        if not line:
            return None
        if line.endswith('\n'):
            line = line[:-1]
        return "%i: %s" % (self.lineno, line)

    def __getstate__(self):
        # Copy the object's state from self.__dict__ which contains
        # all our instance attributes. Always use the dict.copy()
        # method to avoid modifying the original state.
        state = self.__dict__.copy()
        # Remove the unpicklable entries.
        del state['file']
        return state

    def __setstate__(self, state):
        # Restore instance attributes (i.e., filename and lineno).
        self.__dict__.update(state)
```

```
# Restore the previously opened file's state. To do so, we need to
# reopen it and read from it until the line count is restored.
file = open(self.filename)
for _ in range(self.lineno):
    file.readline()
# Finally, save the file.
self.file = file
```

A sample usage might be something like this:

```
>>> reader = TextReader("hello.txt")
>>> reader.readline()
'1: Hello world!'
>>> reader.readline()
'2: I am line number two.'
>>> new_reader = pickle.loads(pickle.dumps(reader))
>>> new_reader.readline()
'3: Goodbye!'
```

Custom Reduction for Types, Functions, and Other Objects

Added in version 3.8.

Sometimes, [dispatch_table](#) may not be flexible enough. In particular we may want to customize pickling based on another criterion than the object's type, or we may want to customize the pickling of functions and classes.

For those cases, it is possible to subclass from the [Pickler](#) class and implement a [reducer_override\(\)](#) method. This method can return an arbitrary reduction tuple (see [__reduce__\(\)](#)). It can alternatively return [NotImplemented](#) to fallback to the traditional behavior.

If both the [dispatch_table](#) and [reducer_override\(\)](#) are defined, then [reducer_override\(\)](#) method takes priority.

Note: For performance reasons, [reducer_override\(\)](#) may not be called for the following objects: `None`, `True`, `False`, and exact instances of [int](#), [float](#), [bytes](#), [str](#), [dict](#), [set](#), [frozenset](#), [list](#) and [tuple](#).

Here is a simple example where we allow pickling and reconstructing a given class:

```
import io
import pickle

class MyClass:
    my_attribute = 1

class MyPickler(pickle.Pickler):
    def reducer_override(self, obj):
        """Custom reducer for MyClass."""
        if getattr(obj, "__name__", None) == "MyClass":
            return type, (obj.__name__, obj.__bases__,
                          {'my_attribute': obj.my_attribute})
        else:
            # For any other object, fallback to usual reduction
            return NotImplemented

f = io.BytesIO()
p = MyPickler(f)
```

```
p.dump(MyClass)

del MyClass

unpickled_class = pickle.loads(f.getvalue())

assert isinstance(unpickled_class, type)
assert unpickled_class.__name__ == "MyClass"
assert unpickled_class.my_attribute == 1
```

Out-of-band Buffers

Added in version 3.8.

In some contexts, the `pickle` module is used to transfer massive amounts of data. Therefore, it can be important to minimize the number of memory copies, to preserve performance and resource consumption. However, normal operation of the `pickle` module, as it transforms a graph-like structure of objects into a sequential stream of bytes, intrinsically involves copying data to and from the pickle stream.

This constraint can be eschewed if both the *provider* (the implementation of the object types to be transferred) and the *consumer* (the implementation of the communications system) support the out-of-band transfer facilities provided by pickle protocol 5 and higher.

Provider API

The large data objects to be pickled must implement a `__reduce_ex__()` method specialized for protocol 5 and higher, which returns a [PickleBuffer](#) instance (instead of e.g. a [bytes](#) object) for any large data.

A [PickleBuffer](#) object *signals* that the underlying buffer is eligible for out-of-band data transfer. Those objects remain compatible with normal usage of the `pickle` module. However, consumers can also opt-in to tell `pickle` that they will handle those buffers by themselves.

Consumer API

A communications system can enable custom handling of the [PickleBuffer](#) objects generated when serializing an object graph.

On the sending side, it needs to pass a *buffer_callback* argument to [Pickler](#) (or to the [dump\(\)](#) or [dumps\(\)](#) function), which will be called with each [PickleBuffer](#) generated while pickling the object graph. Buffers accumulated by the *buffer_callback* will not see their data copied into the pickle stream, only a cheap marker will be inserted.

On the receiving side, it needs to pass a *buffers* argument to [Unpickler](#) (or to the [load\(\)](#) or [loads\(\)](#) function), which is an iterable of the buffers which were passed to *buffer_callback*. That iterable should produce buffers in the same order as they were passed to *buffer_callback*. Those buffers will provide the data expected by the reconstructors of the objects whose pickling produced the original [PickleBuffer](#) objects.

Between the sending side and the receiving side, the communications system is free to implement its own transfer mechanism for out-of-band buffers. Potential optimizations include the use of shared memory or datatype-dependent compression.

Example

Here is a trivial example where we implement a [bytearray](#) subclass able to participate in out-of-band buffer pickling:

```
class ZeroCopyByteArray(bytearray):

    def __reduce_ex__(self, protocol):
        if protocol >= 5:
            return type(self)._reconstruct, (PickleBuffer(self),), None
        else:
            # PickleBuffer is forbidden with pickle protocols <= 4.
            return type(self)._reconstruct, (bytearray(self),)

    @classmethod
    def _reconstruct(cls, obj):
        with memoryview(obj) as m:
            # Get a handle over the original buffer object
            obj = m.obj
            if type(obj) is cls:
                # Original buffer object is a ZeroCopyByteArray, return it
                # as-is.
                return obj
            else:
                return cls(obj)
```

The reconstructor (the `_reconstruct` class method) returns the buffer's providing object if it has the right type. This is an easy way to simulate zero-copy behaviour on this toy example.

On the consumer side, we can pickle those objects the usual way, which when unserialized will give us a copy of the original object:

```
b = ZeroCopyByteArray(b"abc")
data = pickle.dumps(b, protocol=5)
new_b = pickle.loads(data)
print(b == new_b) # True
print(b is new_b) # False: a copy was made
```

But if we pass a `buffer_callback` and then give back the accumulated buffers when unserializing, we are able to get back the original object:

```
b = ZeroCopyByteArray(b"abc")
buffers = []
data = pickle.dumps(b, protocol=5, buffer_callback=buffers.append)
new_b = pickle.loads(data, buffers=buffers)
print(b == new_b) # True
print(b is new_b) # True: no copy was made
```

This example is limited by the fact that [bytearray](#) allocates its own memory: you cannot create a `bytearray` instance that is backed by another object's memory. However, third-party datatypes such as NumPy arrays do not have this limitation, and allow use of zero-copy pickling (or making as few copies as possible) when transferring between distinct processes or systems.

See also: [PEP 574](#) – Pickle protocol 5 with out-of-band data

Restricting Globals

By default, unpickling will import any class or function that it finds in the pickle data. For many applications, this behaviour is unacceptable as it permits the unpickler to import and invoke arbitrary code. Just consider what this hand-crafted pickle data stream does when loaded:

```
>>> import pickle
>>> pickle.loads(b"cos\nsystem\n(S'echo hello world'\ntr.")
hello world
0
```

In this example, the unpickler imports the [os.system\(\)](#) function and then apply the string argument “echo hello world”. Although this example is inoffensive, it is not difficult to imagine one that could damage your system.

For this reason, you may want to control what gets unpickled by customizing [Unpickler.find_class\(\)](#). Unlike its name suggests, `Unpickler.find_class()` is called whenever a global (i.e., a class or a function) is requested. Thus it is possible to either completely forbid globals or restrict them to a safe subset.

Here is an example of an unpickler allowing only few safe classes from the [builtins](#) module to be loaded:

```
import builtins
import io
import pickle

safe_builtins = {
    'range',
    'complex',
    'set',
    'frozenset',
    'slice',
}

class RestrictedUnpickler(pickle.Unpickler):

    def find_class(self, module, name):
        # Only allow safe classes from builtins.
        if module == "builtins" and name in safe_builtins:
            return getattr(builtins, name)
        # Forbid everything else.
        raise pickle.UnpicklingError("global '%s.%s' is forbidden" %
                                      (module, name))

    def restricted_loads(s):
        """Helper function analogous to pickle.loads()."""
        return RestrictedUnpickler(io.BytesIO(s)).load()
```

A sample usage of our unpickler working as intended:

```
>>> restricted_loads(pickle.dumps([1, 2, range(15)]))
[1, 2, range(0, 15)]
>>> restricted_loads(b"cos\nsystem\n(S'echo hello world'\ntr.")
Traceback (most recent call last):
...
pickle.UnpicklingError: global 'os.system' is forbidden
>>> restricted_loads(b'cbuiltins\neval\n'
...                  b'(S\'getattr(__import__("os"), "system")\'
...                  b'("echo hello world")\'\'ntr.')
```

```
Traceback (most recent call last):
```

```
...  
pickle.UnpicklingError: global 'builtins.eval' is forbidden
```

As our examples shows, you have to be careful with what you allow to be unpickled. Therefore if security is a concern, you may want to consider alternatives such as the marshalling API in [xmlrpc.client](#) or third-party solutions.

Performance

Recent versions of the pickle protocol (from protocol 2 and upwards) feature efficient binary encodings for several common features and built-in types. Also, the pickle module has a transparent optimizer written in C.

Examples

For the simplest code, use the [dump\(\)](#) and [load\(\)](#) functions.

```
import pickle  
  
# An arbitrary collection of objects supported by pickle.  
data = {  
    'a': [1, 2.0, 3+4j],  
    'b': ("character string", b"byte string"),  
    'c': {None, True, False}  
}  
  
with open('data.pickle', 'wb') as f:  
    # Pickle the 'data' dictionary using the highest protocol available.  
    pickle.dump(data, f, pickle.HIGHEST_PROTOCOL)
```

The following example reads the resulting pickled data.

```
import pickle  
  
with open('data.pickle', 'rb') as f:  
    # The protocol version used is detected automatically, so we do not  
    # have to specify it.  
    data = pickle.load(f)
```

Command-line interface

The pickle module can be invoked as a script from the command line, it will display contents of the pickle files. However, when the pickle file that you want to examine comes from an untrusted source, `-m pickletools` is a safer option because it does not execute pickle bytecode, see [pickletools CLI usage](#).

```
python -m pickle pickle_file [pickle_file ...]
```

The following option is accepted:

pickle_file

A pickle file to read, or `-` to indicate reading from standard input.

See also:

Module [copyreg](#)

Pickle interface constructor registration for extension types.

Module [pickletools](#)

Tools for working with and analyzing pickled data.

Module [shelve](#)

Indexed databases of objects; uses pickle.

Module [copy](#)

Shallow and deep object copying.

Module [marshal](#)

High-performance serialization of built-in types.

Footnotes

- [1] Don't confuse this with the [marshal](#) module
- [2] This is why [lambda](#) functions cannot be pickled: all `lambda` functions share the same name: `<lambda>`.
- [3] The exception raised will likely be an [ImportError](#) or an [AttributeError](#) but it could be something else.
- [4] The [copy](#) module uses this protocol for shallow and deep copying operations.
- [5] The limitation on alphanumeric characters is due to the fact that persistent IDs in protocol 0 are delimited by the newline character. Therefore if any kind of newline characters occurs in persistent IDs, the resulting pickled data will become unreadable.